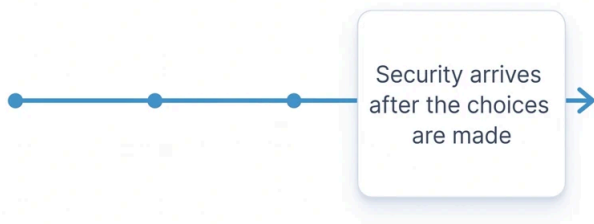


Security by Design and integration into projects

✓ Securing what does not exist yet

Security bolted on at the end



Security by Design



Session objectives

By the end of this lecture:

- **Explain** what Security by Design is, which economic and technical problem it answers, and what it does not guarantee
- **Integrate** security into a real project cycle, placing security milestones with verifiable pass criteria
- **Articulate** the three regimes of a project, buying, developing, having developed, with the requirements that suit each

- **Rework** a MERIDIAN group project from the root to show what the design should have required from the outset

Prerequisites

- This morning's tutorial: the supply chain, the pivot attack and the inventory of the group's third-party dependencies.
- The session 1 governance: the framework PSSI (the group information system security policy) and its directives, the RACI (responsible, accountable, consulted, informed) matrix and the uniqueness of the ultimate owner, the governed right of exception.
- The session 2 mapping: business assets, supporting assets and DICT (availability, integrity, confidentiality, traceability) needs, which provide the starting requirements of any project.
- The framework chosen in session 3 and the session 4 audit report, which say respectively what the group imposes on itself and where it stands.

The problem this lecture attacks

This morning, the tutorial looked at the third parties already installed in the system: the contracted provider, the update flow, the inherited interconnection. But each of these links has a birth point: a project. One day, someone decided to buy that software, to sign that maintenance contract, to open that flow. And in most organizations, security was not in the room that day. It arrives afterwards, when the architecture is frozen, the budget spent, the contract signed: all that is left for it is patching.

The MERIDIAN group carries a well-known illustration of this: the learning platform project of the Éducation subsidiary, connected to Territoires' administrative databases. The Territoires IT manager opposes it, the matter goes up to arbitration, and a DMZ (demilitarized zone, a buffer network) and secured APIs will probably have to be imposed after the fact. Whatever the arbitration, the real cost of the episode is already there: a conflict between subsidiaries, a delay, an architecture to be reworked. All of that could have been bought in one meeting if the requirements had been set at framing.

10,6%

OF THE THREATS RECORDED BY ENISA FALL
UNDER THE SUPPLY CHAIN

ENISA, Threat Landscape 2025

85%

OF COMPANIES ADOPT SECURITY CLAUSES IN
THEIR CONTRACTS TO MITIGATE THIRD-PARTY
RISKS

CESIN / OpinionWay, 2025

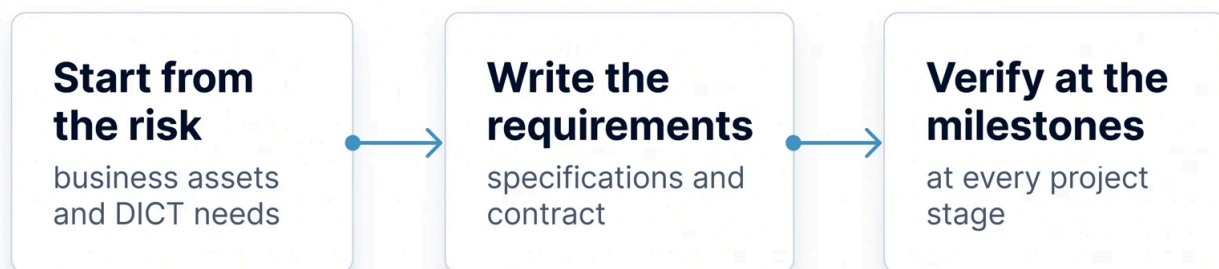
These two figures say the same thing from both ends: the threat comes through what is acquired and what is contracted, and organizations have understood it to the point of massively arming their contracts. The question of this lecture is therefore: how to make security a birth property of projects, and not an end-of-run patch.

Security by Design: a birth requirement, not a finishing coat

First, the notion that gives the sequence its title. The concrete problem has just been seen: security added at the end of a project runs into choices already made. A network designed flat is not easily segmented, strong authentication cannot be added to an API (application programming interface) whose protocol does not carry it, and a contract signed without a security clause is not renegotiated without pain. **Security by Design**, security from the design stage, is the principle by which security requirements are defined, arbitrated and integrated from a project's framing, on the same footing as functional requirements, then verified at each step of its realization. It is neither a technology nor a document: it is a discipline of engineering and governance.

Its mechanics fit in three movements. First movement: start from the risk, not from the catalog. A project begins by identifying what it will handle (which business assets, which data, with which needs for availability, integrity, confidentiality and traceability, the ones session 2 taught how to rate) and what could harm it. The EBIOS Risk Manager guide v1.5 explicitly lists this use among its own: the method serves to « apprécier et traiter les risques relatifs à un projet numérique, notamment dans l'objectif d'une homologation de sécurité » (assess and treat the risks relating to a digital project, in particular with a view to a security authorization). Second movement: convert this risk into dated requirements, written into the project's documents, specifications included, because a requirement that is not written down at the time of the call for tenders will not exist in the contract. Third movement: verify, at each step, that the requirements survive the technical choices, the schedule compromises and the contract amendments, which presupposes checkpoints, the milestones presented in a moment.

Three principles complete the mechanics, and the syllabus names them. Secure defaults: a system delivered closed, opened when needed, rather than a system delivered open and closed after the incident. Least privilege from the design stage: each component and each account only receives the rights of its function, which is decided on the diagram, not in operations. Security debt: each skipped milestone creates a debt, repaid later with interest, as an architecture rework or an incident; a requirement set at framing costs one meeting, the same requirement caught up in production costs a project.



This principle is not a theoretician's elegance: the texts bearing down on the group carry it. The NIS 2 (Network and Information Security) directive imposes, among its minimum risk management measures, "security in network and information systems acquisition, development and maintenance, including vulnerability handling and disclosure" (Article 21, paragraph 2, point (e)). And the framework the group chose in session 3 devotes one control of its Annex A to the question, visible in the tool, in the session 4 audit: A.5.8, « Sécurité de l'information dans la gestion de projet » (information security in project management). In other words: integrating security into projects is not a CISO (chief information security officer) preference, it is a requirement of the adopted framework and an obligation in the making.

Its limit remains, and it is twofold. First, by design exempts nothing in operations: a perfectly designed system degrades from the day it goes live if no one applies the patches or watches the logs; design opens the game, it does not win it. Second, the principle dies from excess as much as from deficiency: a CISO who demands everything, everywhere, from framing onwards turns security into a bureau of refusals, and the business lines quickly learn to go around the bureau. Proportionality is constitutive of the principle: the requirements are calibrated to the project's real DICT needs, not to the maximum imaginable.

Concept sheet: Security by Design

The problem is that security added at the end of a project costs dearly, arrives late and is negotiated from a position of weakness. **The definition** is the principle by which security requirements are defined from a project's framing, integrated into its contractual documents and verified at every step. **The mechanics** are these: start from the risk (business assets, DICT needs, scenarios), convert it into written requirements, check at the milestones. **The limit** is that it is a substitute for neither operations nor proportionality; a maximalist by-design kills the projects it claims to protect.

Security milestones: giving the principle teeth

A principle without a checkpoint is a wish. The operational question is: at which precise moments of the project does security have the right to look, to require and to block? The answer is called the **security milestone**. First, a project milestone in the general sense: a dated crossing point between two phases, where it is decided to continue, adjust or stop. A security milestone is that same crossing point, equipped with explicit security criteria: the project passes the milestone only if the criteria are met or if a motivated exception is granted. Integrating security into projects, the third notion of this lecture, is nothing other than that: the organized presence of security at every phase, from framing to end of life, rather than a single review at the end of the run.

Here is the outline, on a classic project cycle. It holds for in-house development as well as for a purchase, only the deliverables change.

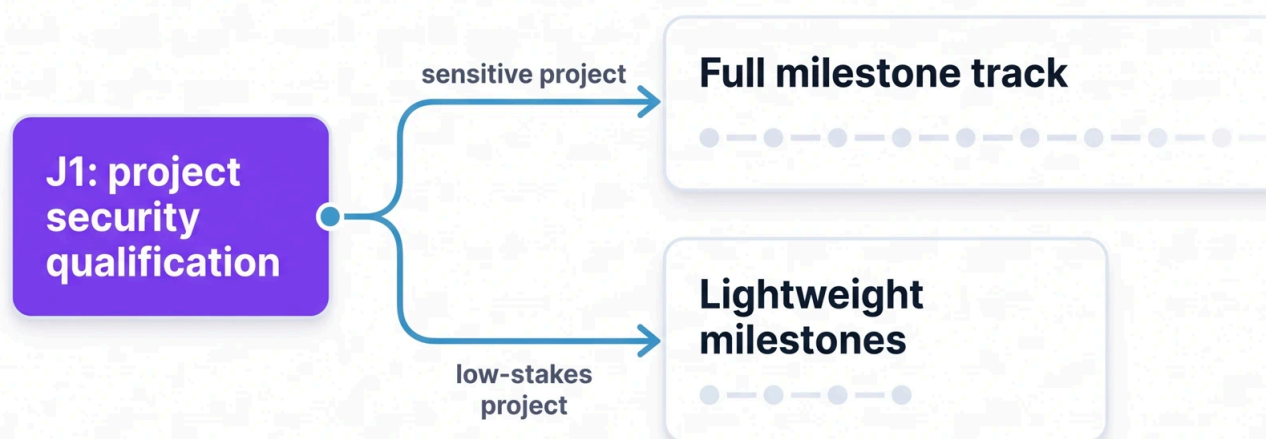
Project phase	Security milestone	Questions asked at the milestone	What is produced
Framing	M1: security qualification of the project	Which business assets and data? Which DICT needs? Which proportionate level of risk analysis?	Project sheet, security section, data classification
Design	M2: architecture validation	Are the requirements translated into the architecture? Segmentation, authentication, logging planned?	Architecture file, security requirements added to the specifications
Realization or purchase	M3: contracting and development	Do the requirements appear in the contract? Does the development follow the security rules?	Contract with security clauses, test evidence
Acceptance	M4: verification before service	Are the requirements demonstrated, not merely declared? Audit or penetration tests if the level requires it?	Security acceptance report, go-live decision
Operations	M5: conditions of life	Who maintains, who monitors, who fixes, within what deadlines?	Security operating conditions, handover to the subsidiary CISO
End of life	M6: clean exit	Data returned or destroyed? Access revoked? Reversibility executed?	End-of-life report

This table is the afternoon's working tool: the project sheet of deliverable D6 takes up its logic. Three comments to read it as a professional. First, each milestone has an owner: the session 1 governance applies as is, a single ultimate owner per decision, and the CISO is not necessarily that owner; the CISO is the one who prepares and attests. Second, a milestone only makes sense equipped with a verifiable pass criterion: "security has been taken

into account" is not a criterion, "the DICT needs are rated and the data classification is validated by the business" is one.

Finally, blocking must be a genuinely possible outcome, otherwise the milestone is a ritual: this is where the exception mechanism the group gave itself comes in, an exception to the framework PSSI is motivated and validated by the Group CISO within 48 hours, it leaves a trace and a deadline, whereas forcing through leaves only a precedent. For systems subject to security authorization, the one worked on in session 4, milestone M4 takes a regulatory form: the security authorization decision is the pass criterion.

The limit of milestones has a one-word name: friction. Each milestone costs projects time, and a mechanism that project managers experience as a tollbooth gets bypassed. The countermeasure is not to lighten security but to proportion it: every project passes M1, and it is M1 that decides the thickness of the following milestones according to the risk. An internal display board with no sensitive data does not deserve the same course as a patient record management system.



✓ Buying, developing, having developed: three regimes, three sets of requirements

A project does not always build: often it buys, or it has someone else build. The third-party question, opened this morning, thus returns here through the projects door, and that is deliberate: the best control of third parties begins before the signature. The three regimes do not call for the same reflexes.

Regime	What the group controls	What the group does not control	Main security lever
Develop in-house	The code, the architecture, the schedule	The security skills available in-house	Development rules, tests, code review
Buy a product or service	The choice, the configuration, the contract	The vendor's code, its build chain, its patch schedule	Requirements in the call for tenders, supplier evaluation, clauses
Have developed or outsource operations	The requirements, the oversight, the reversibility	The day-to-day execution, the provider's staff	Contract, Security Assurance Plan, audits, contractual milestones

The common thread of the three rows: in every case, what was not required at the start can only be begged for afterwards. The NIS 2 directive pushes this reasoning all the way into the suppliers: when an entity chooses its supply chain measures, Article 21, paragraph 3, requires it to take into account "the vulnerabilities specific to each direct supplier and service provider and the overall quality of products and cybersecurity practices of their suppliers and service providers, including their secure development procedures". Read closely: the law will ask for an evaluation of how suppliers develop.

Annex A of the chosen framework covers the same ground in the language of controls; the wordings below follow the tool's library: supplier relationships (A.5.19), security within supplier agreements (A.5.20), the information and communication technology supply chain (A.5.21), and, on the building side, the secure development life cycle (A.8.25) through to outsourced development (A.8.30). These controls are not to be learned by heart: what is to be remembered is that each row of the table above corresponds to a family of controls already provided for by the group's framework.

A word on the most modern side of development, because the group will come to it: continuous integration and deployment chains. ANSSI (the French national cybersecurity agency) devotes a factsheet to it, « Les essentiels DevSecOps », which defines the approach as « une méthodologie qui vise à inclure les pratiques de sécurité dans le processus de développement et de mise en production d'applications » (a methodology that aims to include security practices in the process of developing and releasing applications), and which notably recommends « faire une analyse de risque globale en prenant en compte les chemins de compromission des postes des développeurs, de la sous-traitance, de la chaîne CI/CD » (carrying out a global risk analysis that takes into account the compromise paths of developer workstations, subcontracting and the CI/CD chain) and « gérer les dépendances avec rigueur : les minimiser, les évaluer et appliquer les correctifs de sécurité avant déploiement » (managing dependencies rigorously: minimizing them, evaluating them and applying security patches before deployment). The pattern is recognizable: the software factory line is itself a supply chain, and the 2020 SolarWinds case showed what it costs when it is the vendor's factory that is compromised. By design therefore does not stop at the product: it covers the product's factory.

✓ The MERIDIAN case: reworking the learning platform from the root

The whole lecture, applied to the episode that opened it, : Éducation's learning platform, connected to Territoires' administrative databases. It is the project the D6 sheet will handle this afternoon for the Éducation and Territoires cluster; Santé and Logistique will apply the same grid to the project in their own pack.

The project, replayed with milestones. At M1, the qualification would have laid out the facts in one page: the project handles student identities and data drawn from local authorities' administrative databases, regulated citizen data; the sponsoring subsidiary has a tight budget and a heterogeneous estate of third-party software; the connection crosses a subsidiary border, hence a border of responsibility, exactly the type of flow that the session 2 mapping places in the ecosystem view. That one page was enough to trigger two consequences: a high level of requirement on authentication, the group's framework PSSI directive already imposing MFA (multi-factor authentication), and the invitation of the Territoires IT manager to the framing table, since their data was at stake.

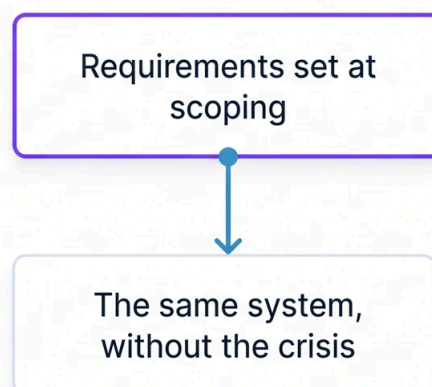
At M2, the architecture would have carried from the outset what the arbitration will have to impose after the fact: a DMZ, secured APIs, segmented flows. At M3, if the platform is bought or developed by a third party, the requirements pass into the contract, with what the next tutorial will teach how to write: incident notification, audit rights, reversibility. At M4, acceptance testing demonstrates, evidence in hand, that unplanned flows are blocked. At M5, the question "who reads the platform's logs, and where" receives an answer before go-live, and the session 4 audit established that Territoires still keeps its logs locally: the project was the opportunity to fix that, not to reproduce it.

To compare with what is actually happening: a manager's opposition, a governance arbitration under way, measures that will be imposed downstream. The final technical result will be close; the organizational cost is nothing alike. That is the general lesson of Security by Design: it does not produce different systems, it produces the same systems without the crisis, and it turns the CISO from a last-minute opponent into a co-designer.

Without milestones



With milestones



One last point of intellectual honesty, because a CISO trained at Master's level must know how to phrase it before a management team: designing securely does not mean predicting every scenario. This morning still, the analysis stopped at "here are our dependencies"; the workshops in the rest of the morning teach precisely how to tell how an attacker would cross the ecosystem, and thus how to feed milestones M1 and M2 with scenarios rather than intuitions. Design rests on risk analysis; and risk analysis, at MERIDIAN, is the method begun yesterday and continued in an hour.

✓ What to remember before the tutorial

Three notions are set out and articulated. Security by Design: security requirements are born with the project, carried by the risk analysis, written into the documents, contractual ones included. Security milestones: six crossing points, each equipped with verifiable criteria and a single owner, with the motivated exception as the governed relief valve. Integration into projects: security present at every phase, proportioned by the first milestone, and extended to the three regimes, developing, buying, having built, all the way to the suppliers' development practices, as regulation will require. The tutorial that follows takes the third regime seriously: when the third party is durably installed in the system, IT outsourcing, how is it analyzed and how is the contract written. And the afternoon will convert it all into deliverables: the scenarios in the tool, then the project sheet and the requirements of D6.